EURAM 2011
11th
1st-4th June 2011, Tallinn
Management Culture in the 21st Century
EBS, Estonian Business School (Tallinn, Estonia)

Improving management and tools for project performance in software development: a system dynamics approach

Author: STEFANO ARMENIA - Email: armenias@mac.com
University: ROME "LA SAPIENZA" UNIVERSITY

Track: 37. Research Methods & Research Practice General Track

Co-author(s): Massimilianoi Schiraldi (Università di Roma "Tor Vergata")
/ **Diego Falsini** (Università di Roma "Tor Vergata")
/ **Daniele Canini** (Università di Roma "Tor Vergata")

Access to this paper is restricted to registered delegates of the EURAM 2011 (European Academy of Management) Conference.

Estonian Business School
EIASM
EURAM

Improving management and tools for project performance in software development: a system dynamics approach

Stefano Armenia[1], Massimiliano M. Schiraldi[2], Diego Falsini[3], Daniele Canini[4]

Abstract
While existing research has mainly focused on project management's static view, our work investigates the impacts of projects' structure and behavioural dynamics on their performance, with a specific focus on the influence of some peculiar development processes. A dynamic simulation model of a single-phase project was built using the system dynamics methodology. The model integrates several previously developed and tested project structures and adds a separate structure for the negotiation process. Simulations describe the behaviours generated by the interaction of customized development processes in single-phase projects. Project performances are measured in terms of time, quality and cost. Our research aims to show that development processes, as well as shared resource levelling techniques, significantly impact the dynamic behaviour of projects through the feedback, delays and nonlinear relationships which are usually omitted in traditional project management practice, as well as in methods, tools and models, but are very important descriptors of project complexity. Expanding the models used to manage projects to include dynamic features requires a change of focus by researchers and practitioners. The system dynamics methodology provides some of the tools for developing and implementing such an expansion in project models.

Keywords
Project Management, System Dynamics, Resource Allocation, Decision Support Systems, Collegial Decision Making

1. Introduction

The combination of feedback, time delays, and nonlinear relationships in project structures have been shown to reduce performance and cause them to be very difficult to manage (Thomas and Napolitan, 1994; Reichelt, 1990; Cooper, 1980). The dynamic nature of project behaviour precludes the generation of a single set of decision rules which are robust in the

---

[1] Corresponding author. "CATTID" - Sapienza University of Rome, P.le Aldo Moro, 5 - 00185 Rome (Italy) tel. +39.333.8711021. E-mail address: armenia@cattid.uniroma1.it

[2] "Tor Vergata" University of Rome, Dept. of Enterprise Engineering, Rome, Italy. E-mail address: schiraldi@uniroma2.it

[3] "Tor Vergata" University of Rome, Dept. of Enterprise Engineering, Rome, Italy. E-mail address: diego.falsini@uniroma2.it

[4] "Tor Vergata" University of Rome, Dept. of Enterprise Engineering, Rome, Italy. E-mail address: dc.isci@gmail.com

face of all possible project conditions. As a matter of fact, both complexity and dynamic features of projects seems to be poorly understood by managers (Diehl and Sterman, 1995; Sterman, 1994; Paich and Sterman, 1993; Rechtin, 1991). In addition traditional tools are inadequate for dealing with the dynamic complexity of projects.

Under this point of view, System Dynamics can play a key and strategic role in risk management, that "is an essential process of every software risk management plan. Several risk assessment techniques are based on the subjective judgement of domain experts. Subjective risk assessment techniques are human intensive and error-prone" (Sherif M. Yacoub, Hany H. Ammar, 2001). Thus entrusting a key strategic step only to manager intuition is a source of low technicality and can generate a proliferation of harmful events that, in a wider perspective, might be avoided for the project.

A project should be really considered as a man-made goal-oriented open system and, thus, it tends to be unpredictable and unstable. The complexity of projects and of their environment has increased the disruptive effect of subjective human factors. Again, personal judgement based on past experience is no longer sufficient to cope with this problem. There is a need to better understand the strategic activity of project management and to learn effectively from past failures; this can only be achieved through a more formal systemic analysis and by means of a methodological approach.

System Dynamics (SD) may be useful in describing causal and dynamic complexity arising in software projects and organizations, thus eventually allowing for the building of a new set of tools which will support management in decisions as well as allow them to experiment in learning environments and checking out their hypotheses without implementing them first and wait for the, rather often, catastrophic consequences.

Research practice has demonstrated that the methodological approach of system dynamics is more suitable to strategic decisions than to operative ones, since (Senge P., 1992) "it aims at taking into account not just the single elements of a system, rather the sum of its parts intended as a whole, by concentrating on their relationships rather than on elements considered separately". The most fascinating conclusion is then that there are margins for improvement by considering the synergy of traditional PM techniques with the SD approach, in order to assign operative planning activities to the first ones and strategic decisions based on the dynamic and feedback structure intrinsic to managing projects (with particular reference to greater complexity typical of software projects) to the second one.

2. Literature review

2.1 Dependencies and feedback problems of development projects in literature

Traditional project management models based on the Critical Path Method (CPM) and PERT (Moder et al. 1983; Halpin and Woodhead 1980) provides several tools for trading away good performance in one measure for improved performance in another. For example durations of activities along the critical path can be shortened by adding more resources (Ulrich and Eppinger, 1994; Wheelwright and Clark, 1992; Moder et al., 1983). The effects of altering activity dependencies among activities to shorten the critical path can be investigated (Barrie and Paulson, 1984; Moder et al., 1983). These methods are limited by their use of an indirect project measure (time) and by bundling the characteristics of and

relationships among scope, resources, and processes in each activity into a single duration estimate. They also tend to ignore iteration or require that iteration be implicitly incorporated into duration estimates and precedence relationships.

More sophisticated models based on a joint CPM/PERT paradigm address some of these limitations, but cannot fully model development processes. Other research approaches identify some dynamic consequences of different project structures on project performance (Smith and Eppinger 1997; Eppinger et al. 1994; Steward 1981). From relevant literature and case studies (Kim E. van Oorschot, Kishore Sengupta, Luk N. van Wassenhove (2009), Austin Robert D. (2001), Lindstrom, Lowel, Ron Jeffries (2004), Molokken_Ostvold, Kjetil, Magne Jorgensen (2005), Meso Peter, Radhika Jain (2006), Cao Lan, Balasubramanian Ramesh (2008)), it is possible to state that the "reality of software projects always let the manager cope with frequent delays and budget overheads, which inevitably lead to problems in the functionality and quality of the developed software/product".

The main issue to deal with is to understand which are the origins of the delay & disruption effects and on delivery time slippage, in order to identify the correct management leverages on which to act in order to mitigate those problems. Since the underacting dynamics are archetypical of every project, a general representation of a typical GANTT diagram (both the WP and the Task representations display the same structures) helps understand the underlying structure and hence the fundamental methodological role that the SD may exert.

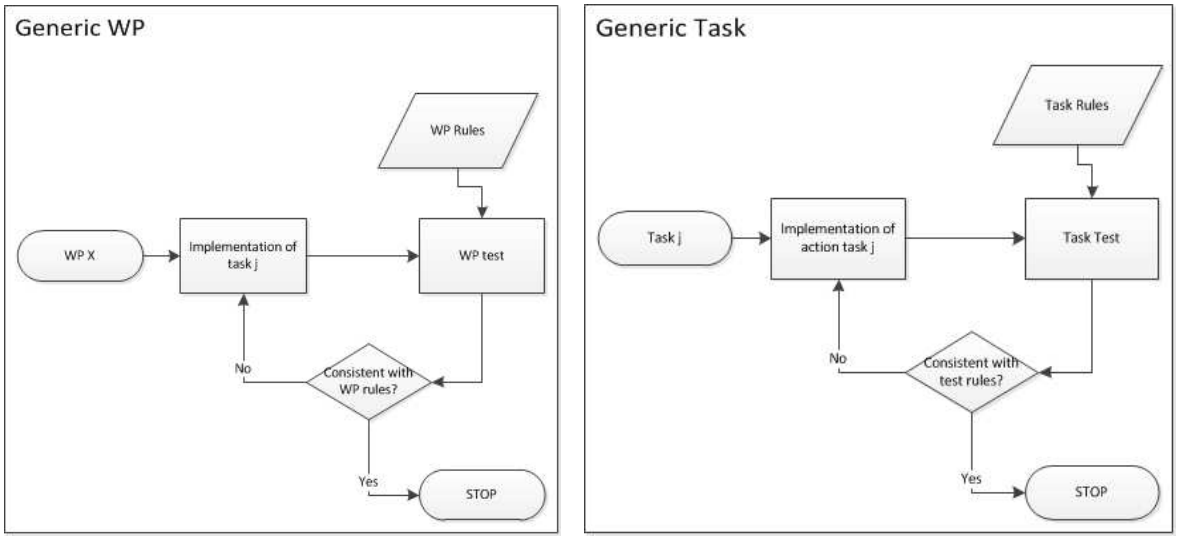

Figure 1: Feedback loops at WP and single task level

From a systemic perspective, it is thus easy to understand, just by considering the underlying archetypical structures of software projects that issues arise by the fact that classical PM tools and methodologies fail to address the feedback cycles that are intrinsic to the software life-cycle. The existence of such feedback loops can be inferred by analysing the diagrams in figure 1, where acceptance rules effectively create rework cycles at different levels (at the WP level but also at the single task level). Under a structural point of view, this translates into nested rework cycles that contribute to make things even more complex and complicated. This easily helps understand how a linear approach, which is typical of actual PM practice, may lead to underestimation of possible effects and situations during the development of the project, whose behaviour is instead highly non-linear and may thus introduce heavy delays and disruption effects. The situation is even worse if we consider that the above described

structure is not only relevant to the iterations happening at the level of single project-phase, rather also among different project phases.

Reductions in cycle time can be achieved by applying concurrent development techniques (Wheelwright and Clark, 1992, Womack et al., 1990; Nevins and Whitney, 1989), but the cycle time reduction comes at the cost of increased complexity. Steward (1981) and Eppinger et al. (1990) developed the Design Structure Matrix (DSM) to investigate the iterative nature of product development. Design Structure Matrices have been used to map (Smith and Eppinger, 1991) and predict (Morelli and Eppinger, 1993) information flows among activities, but the DSM cannot directly model the structure of a development process over time. Other model structures based on project characteristics have been suggested (Rodrigues and Williams, 1996) and described conceptually (Cooper 1980).

However, the specific features and characteristics that distinguish different development processes have not been described at a formal level of detail as models. Excluding phase-specific development processes, structures from project models implicitly assumes that those development processes have no impact on project performance. Yet the availability of new work due to rework (or as described by eventual precedence relationships) within and between phases is an important constraint on project performance (Rosenau and Moran 1993; Clark and Fujimoto 1991; Wheelwright and Clark 1992; Moder et al. 1983).

Under this point of view, recurring to the SD approach does not only imply to be able to predict “rare” episodes (as a matter of fact, delay and disruption are not “rare” behaviours in all projects), rather, by means of causal loops diagrams and project models simulations, it helps managing available project resources in an efficient way so to be able to meet scheduled deadlines.

2.2 Contribution of System Dynamics to Project Management issues in literature

A first model was proposed by Roberts (1974) to explore the basic dynamics of R&D projects where the concepts of perceived progress and real progress were first introduced. This model was further improved by Kelly (1970) to consider the management of concurrent projects. The model developed by Cooper (1980) at Pugh-Roberts Associates was the first major practical application of SD to Project Management. Richardson and Pugh (1981) presented a model for the management of R&D projects, which summarises the basic feedback structures of the project management process. Abdel-Hamid and Madnick (1991) applied SD in particular to the software development process for the first time. The models proposed by Lin and Levary (1989) considers an explicit breakdown of the project work into the classic life-cycle stages, providing a more detailed analysis for the schedules, budgets and staff allocation to the project. The above developments represent important contributions for the application of SD to software project management. They introduce valuable concepts and ideas that should be considered in the future. However, most of the reported cases refer to post mortem analysis. Here, the model is used to reproduce the behaviour of completed projects and helps to investigate the causes for deviations.
From the review of the literature we identified several key feedback structure on which we developed our model: the rework structure (Figure 1), the labor structure, the schedule structure, the available work structure, the quality structure, the scope structure.

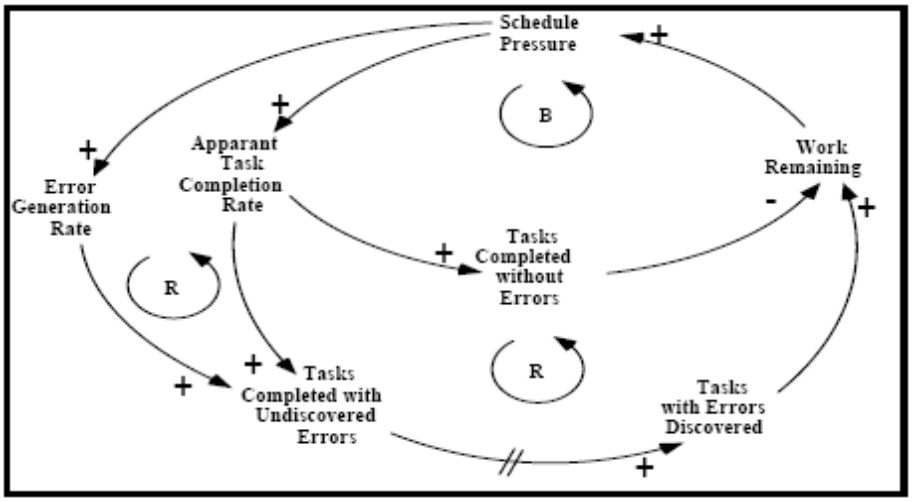

Figure 2: The rework feedback structure (Ford, 1995)

The balancing loop in Figure 2 represents the intended impact of a management response to an increase in schedule pressure, that is reducing the remaining work to be done. The two reinforcing loops represent the impacts of the unintended side effects of the structure, the generation of additional errors which require correction (SD traditional notation, symbols and lexicon are not recalled in this paper, refer to Sterman, 2000).

3. Proposal of a model

3.1 Setting the context

For our goal, which is to show and understand the major dynamics which affect software project management, we will limit our analysis to a single phase (coding) of a mono-product software project, that is a portion of its life cycle, spanning over a maximum of 12-24 months (a medium time-dimension for software projects), and we will assume the development team is fully (100% of time) committed on a single project. We will design our model by taking into considerations the main performance indicators of project management, i.e. related to Cost, Time and Quality issues of the released finished product (or a "work in progress" element released to the subsequent phase), and other aspects like Project Complexity, Uncertainty and Risk.

3.2 The model structure: the coding phase of a software project

The model is composed by four subsystems: development processes, resources, control and productivity. The development processes subsystem is the focus of this work. Ford (1995) describes the other subsystems in detail.

3.2.1 The development process structure and Scope subsystems

Our model uses three features to describe the development process in a single phase: circular iteration, multiple development activities, and dynamic concurrence. Circular iteration is

described with the stock and flow structure (Figure 3).

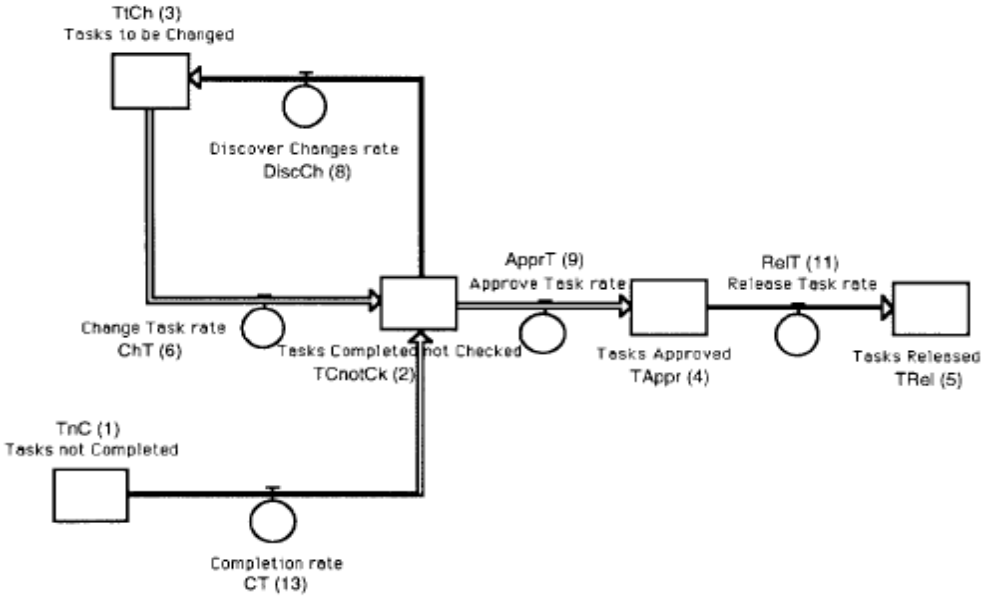

Figure 3: Stock and Flow structure of a single-phase development process (Sterman-Ford, 2000)

We assume that development tasks go through five states: Tasks not Completed (TnC), Tasks Completed by not Checked (TCnotCk), Tasks to be Changed (TtCh), Tasks Approved (TAppr) and Tasks Released (TRel). Tasks initially reside in the Tasks not Completed stock. The first development activity is called "Complete Tasks" (CT). Completed tasks accumulate in the "Completed not Checked" stock. If no tasks require changes or those changes are not discovered during quality assurance, the tasks leave the Completed not Checked stock and pass through the Approve Tasks (ApprT) flow into the stock of Tasks Approved (TAppr). Approved tasks are subsequently released through the Release Tasks (RelT) flow to the stock of Tasks Released (TRel). This represents delivering tasks to the managers of downstream phases or to customers. Tasks needing changes are discovered through the Quality Assurance (QA) activity.

These tasks move through the Discover Changes (DiscCh) flow from the Completed not Checked stock to a stock of Tasks to be Changed. These tasks are corrected or improved through the Change Tasks (ChT) activity and returned to the Completed not Checked stock. Changes can be generated during both completion and correcting or improving tasks.

We formally model the process structure for our single phase with five equations represented in Figure 3 and other 13 equations represented in Figure 4.

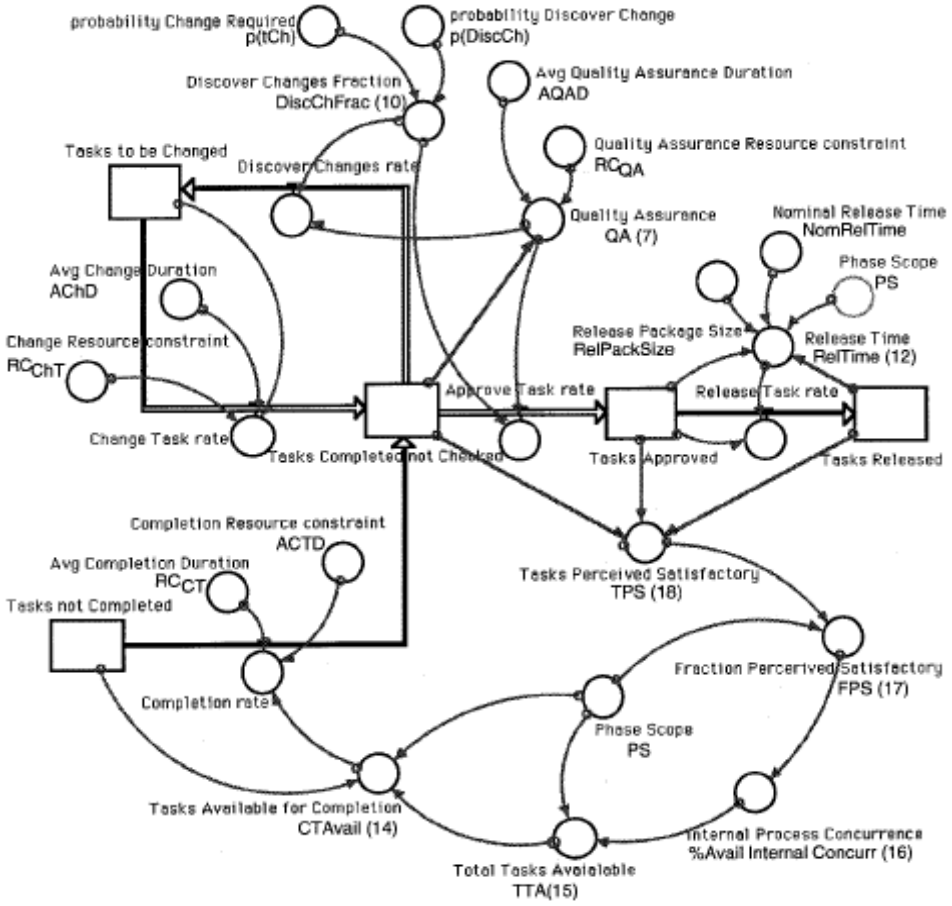

Figure 4: Extended S&F structure with auxiliaries (Sterman-Ford, 2000)

3.2.2 The labor and productivity subsystem

In the previous section we have seen that development of software in a software project phase go through three main activities: (1) Task Completion, (2) Task Review, (3) Task Rework. For each of these activities we have seen in the development sector that there is one or more associated flows of tasks. Such a rate represents the activity process itself, that is represents the "speed" at which tasks are processed. In these sense, such activities processing is constrained by the resources allocated to them. Keeping things simple, we saw that the work rate for each activitiy is equal to the minimum between the rate determined by the availability of work (divided by the average activity duration) and the rate determined by the resources allocated to that activity. This means that resources determine the maximum speed at which an activity may be performed.

3.2.3 The resources subsystem

The project's total Workforce is assumed to be composed by two different workforce elements, namely NewPeople and Experts. In particular, the Average Assimilation Delay in the model has been set to 17 wwk (working week), so almost 85 working days (wday). We divided the workforce into these two categories first because New Hirees almost always pass through an orientation during which they are less than full productive (their weekly overall

productivity has been assumed, as it will be seen in the following, half of that of an expert), both if they have been recruited from outside the company and also if they have been transferred from another project. Second, because we wanted to capture also the training overhead involved in adding new members to a software project. The training of newcomers is often carried out directly "on the job" by veterans, which carries away part of the time that an Expert may in effects more productively (in terms of instant project productivity) allocate on development activities.

3.2.4 The control and plan subsystems

Any control function has at least three elements:

1. Measurement of what is happening in the activity being controlled
2. Evaluation by comparison with expected values
3. Communication of the gap for behaviour control if the need arises for doing so

In a software project (and thus also in a single phase), progress is measured by the number of resources consumed (effort and/or time), tasks completed or both.

3.2.4 The performance subsystems

As has been previously stated, a project performance is mainly evaluated through 3 parameters: Time, Quality and Costs. We have developed an index factor for each one of these three elements, which altogether contribute to an overall Project Performance Index:

$$\text{Project Performance} = (\text{Budget Index} + \text{Final Quality} + \text{Time Index}) / 3$$

3. A case study

Original case study:

- Project Size: 94.100 SLOCs (Single Lines Of Code)
- Estimated effort: 5.000 mandays (Actual: 7.000)
- Estimated Duration: 85 wwk (actual: 110)

We initialized the model with the data from the original case study, taking also into considerations an added factor as the work obsolescence. The latter was considered to provide an added work, over time, of 25% the total initial project size.

- Project Size: 94.100 sloc + 23.000 sloc from the obsolescence factor
- Initial estimated Effort: 3.000fte (full time equivalent)
- Actual Effort Spent: 8942 fte
- Estimated project duration: 85 wwk (with a 30% safety pct, released in only one burst at 90% of the project completion)
- Actual simulated duration: 110 wwk

Overall Performance was attested on a 61%, taking into account costs, time and quality, with the following indexes:

- Time Index: 77,27%
- Budget Index: 35,99%
- Quality Index: 71,00%

The obtained results are in accordance to those which were obtained in the original case-study.
A set of input parameters relating to policies for human resource and allocation to activities management, development process management, costs, quality and time management have been identified:

a. Costs section: Safety Cost, Budget effect on Basework
b. Human Resource section: Willingness to change workforce, Workforce needed calculation method (accounting for effort/time), Maximum/Minimum desired WF levels, Training
c. Development section: Internal Precedence Relationship (Concurrency Function – technology related), Package release size (release policy)
d. Quality section: Target Quality of the project, Effect of perceived quality on QA allocation (quality gap)
e. Allocation section: initial percentages to activities allocation, effect of Schedule Pressure, Quality Gap and Budget on QA and Basework, Desired Rework delay
f. Time section: Safety time, Forecast of completion date calculations, Release slippage to staff function
g. Client Relationship Management section: Communication of Project progress, Negotiation on change requirements, Client's trust.

Experimentation with most of them have showed interesting results which may help management in understanding the underlying dynamics which are affecting project performance as well as allow management to experiment with different policy parameters in order to correct their mental models according to the way they manage projects.

4. Conclusions

This paper addressed the important issue of the causes of dynamic behaviour in software development projects by building, testing and applying a dynamic simulation model of a single-phase project.
Feedback, delays and nonlinear relationships were useful in describing the drivers of dynamic behaviour. The concept of software development as a set of interactive demand-driven activities was used to build a rich description of causal relationships based on previous research. The strong direct and indirect influences of development processes were identified by explicitly separating development processes from resources, scope, and targets and by identifying iterations into and among project phases due to the intrinsic structure of projects.
The research identifies a gap between current project models used for management and the complexity of project structures. A failure to bridge this gap is the reason for which project performance improvement is still limited. Expanding the knowledge and the understanding of project dynamics is a critical part of meeting this need and the development of new or improved tools, as simulation (dynamic, discrete and/or hybrid) for communication and management practice is also expected to be essential to translating improved knowledge and understanding into improved performance.

In fact, it was shown that delays due to development iterations are intrinsic to all projects'

structures and, even if the main IT companies are making a daily use of traditional project management techniques and tools, there has not been any improvement until nowadays that brought to tangible advantages in order to mitigate (or at least try to assess at the project kick-off) the time and cost overheads. The most advanced managerial practices today use optimization and modelling methodologies or tools that are however only able to take into account direct effects of measured variables related to a project, while indirect effects have been shown to be usually those who impact most on project performances. It is then natural to assess that if the delays and disruption problem persist even by recurring to advanced techniques, logical thinking would bring to consider the following: either the techniques used to assess the expected values on time and costs are wrong or, rather, the considered variables are not sufficient to describe the dynamics, the more complex structure and, ultimately, the behaviour of the whole system.

This paper stigmatise the second assumption and postulates the need for a new approach to methodology in project management practice and for research in order to develop new tools able to assess the feedback structures intrinsic to project management systems (as the examples in Figure 5 shows).

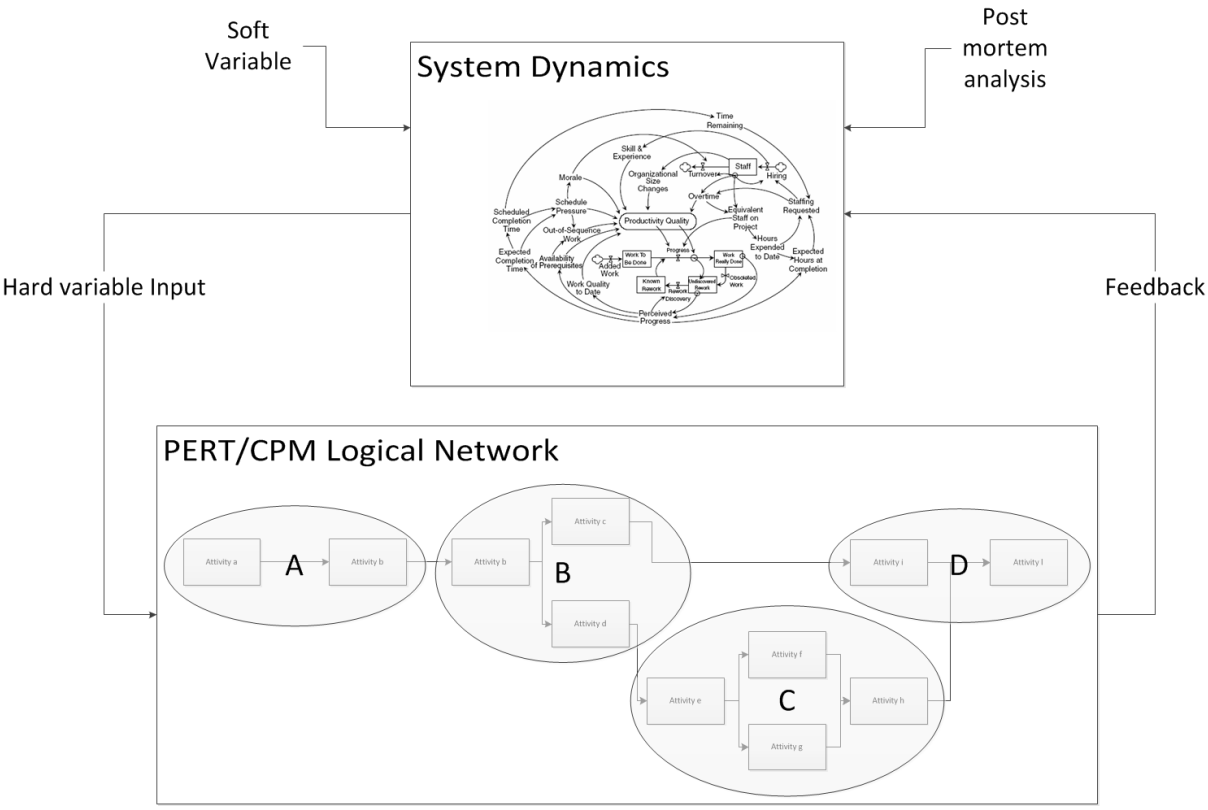

Figure 5: A possible evolution in Project Management methodology and tools.-

As a conclusion, this paper has led to insights concerning the dynamics of projects, a tested framework for modelling projects based on demand for development activities, a tool for future research and a tool for improving the understanding of product development practitioners. This work has created opportunities for expanding the study of project dynamics in several potentially valuable directions and has pushed project management toward a broader image of projects and its role in project performance. It points to ways of improving performance through improved understanding of project structure and behaviour.

Future research will expand and refine the understanding and use of dynamics to manage projects. In particular, the authors propose the extension of the model to a multi-phase software project and to a multi-project environment; the development of standard SD-based project evaluation metrics; the integration at the operational level of our SD model with traditional project modelling approaches.

References

Abdel-Hamid, T., Madnick, S.E., (1991), Software Project Dynamics An Integrated Approach, Prentice Hall (Englewood Cliffs, NJ)

Barrie, D.S., Paulson, B.C., (1984), Professional construction management, McGraw-Hill, (New York)

(6) Cao Lan, Balasubramanian Ramesh (2008), Agile requirements engineering practices: an empirical study. IEEE software.

Clark, K.B., Fujimoto, T., (1991), Product Development Performance Strategy, Organization and Management in the World Auto Industry, Harvard Business School Press (Boston, MA)

Cooper, K. G., (1980), Naval ship production: A claim settled and a framework built. Interfaces 10(6), 20-36.

(2) Austin Robert D. (2001), The effects of the time pressure on quality in software development. Information system research

Diehl, E., Sterman, J.D., (1995), Effects of Feedback Complexity on Dynamic Decision Making, Organizational Behavior and Human Decision Processes, Elsevier

Eppinger, S.D., Whitney, D.E., Smith, R.P., Gebala, D.A., (1994), A model-based method for organizing tasks in product development, Research in Engineering Design, Springer

Ford, N.D., (1995), The dynamics of project management: an investigation of the impacts of project process and co-ordination on performance, Unpublished PhD thesis, Massachusetts Institute of Technology (Cambridge, MA)

Forrester, Jay W., (1961), Industrial Dynamics. Portland, Oregon: Productivity Press

Halpin, D.W., Woodhead, R.W., (1980), Construct Management, John Wiley & Sons (New York)

Kelly, J.C., (1987), A comparison of four design methods for real-time system, Proceedings of the 9th international conference on Software Engineering table of contents, Monterey, IEEE Computer Society Press (Los Alamitos, CA, USA)

Kerzner, H.,(2006), Project management: a systems approach to planning, scheduling, and controlling, Wiley

(1) Kim E. van Oorschot, Kishore Sengupta, Luk N. van Wassenhove (2009), Dynamics of agile software development

(3) Lindstrom, Lowel, Ron Jeffries (2004), Extreme programming and agile software development methodologies. Information system management.

(5) Meso peter, Radhika Jain (2006), Agile software development : adaptive systems principles and best practices. Information system management.

Moder, J.J., Phillips, C.R., Davis, E.W., (1983), Project Management with CPM, PERT and Precedence Diagramming, Van Nostrand Reinhold, (New York)

(4) Molokken_Ostvold, Kjetil, Magne Jorgensen (2005), A comparison of software project overruns-flexible versus sequential development models. IEEE transaction on software engineering

Morelli, M.D., Eppinger, S.D., Gulati, R.K., (1995), Predicting technical communication in product development organizations, Engineering Management, IEEE Transactions on

Nevins, J.L., Whitney, D.E.,(1989), Concurrent Design of Product and Process, McGraw

Hill (New York)
Paich, M., Sterman, J.D., (1993), Boom, Bust, and Failures to Learn in Experimental Markets, Management Science 39(12), 1439-1458
Rechtin, E., (1991), Systems Architecting: Creating and Building Complex Systems, Prentice Hall
Reichelt, K. S., (1990), Halter Marine: A Case Study in the Dangers of Litigation, Technical Report D-4179, System Dynamics Group, MIT Sloan School of Management (Cambridge, MA)
Richardson, G.P., Pugh III A.L., (1981), Introduction to System Dynamics Modeling with Dynamo, MIT Press (Cambridge, MA)
Roberts, E.B., (1974), A simple model of R&D project dynamics. In E. B. Roberts, (ed.), Managerial Applications of System Dynamics, Productivity Press (Cambridge, MA)
Rodrigues, A.G., Williams, T.M., (1996), System dynamics in software project management: towards the development of a formal integrated network. European Journal of Information Systems 6, 51-56
Rosenau, M.D., Moran., J., (1993), Managing the Development of New Products, Achieving Speed and Quality Simultaneously Through Multifunctional Teamwork, Van Nostrand Reinhold (New York)
(9) Senge P., (1992), The Fifth Discipline: The Art and Practice of the Learning Organization, Sperling & Kupfer
(7) Sherif M. Yacoub, Hany H. Ammar (2001), A Methodology for Architectural-Level Reliability Risk Analysis. Publishing Systems and Solutions Laboratory (HP Laboratories Palo Alto).
Smith, R.P., Eppinger, S.D., (1997), Identifying Controlling Features of Engineering Design Iteration, Management Science, JSTOR
Smith, R.P., Eppinger, S.D., (1997), Identifying Controlling Features of Engineering Design Iteration, Management Science, JSTOR
Sterman, J., (2000), Business Dynamics: Systems Thinking for a Complex World. Irwin/McGraw-Hill
Sterman, J.D., (1994), Learning in and about complex systems, System Dynamics Review 10 (2-3), 291-330
Steward, D.V., (1981), The design structure system- A method for managing the design of complex systems, IEEE Transactions on Engineering Management
Ulrich, K.T., Eppinger, S.D., (1994), Methodologies for Product Design and Development, prepublication draft, McGraw-Hill (New York, NY)
Wheelwright, S.C., Clark, K.B., (1992), Revolutionizing Product Development, Quantum Leaps in Speed, Efficiency, and Quality, The Free Press (New York)
Womack J.P., Jones, D.T., Roos, D., (1990), The Machine that Changed the World, Rawson Associates, (New York)